

Kata Team Prämien vom 11.06.2025

Zeitplan

- 10:00 Kata vorstellen und kurz besprechen
- 10:15 Katas beginnen
- 11:45 Mittagspause (wer gemeinsam will... muss nicht)
- 15:00 Lösungen vorstellen und besprechen
- 16:30 Ende der Kata

Focus of the Kata

- Open/Closed Principle (OCP)
- Composition over Inheritance
- Lesbarkeit
- TDD

Focus Explained

Open / Closed Principle (OCP)

The **Open/Closed Principle (OCP)** is one of the five SOLID principles of object-oriented design. It states that software entities (such as classes, modules, and functions) should be **open for extension but closed for modification**. This principle aims to create systems that are easy to extend and maintain without changing existing code.

Key Concepts of the Open/Closed Principle

- **Open for Extension:**
 - You should be able to add new functionality to a software component without altering its existing code. This is often achieved through mechanisms like inheritance, interfaces, or abstract classes.
- **Closed for Modification:**
 - Existing code should not be changed when new functionality is added. This reduces the risk of introducing bugs into already tested and stable code.

Benefits of OCP

- **Improved Maintainability:** By avoiding changes to existing code, you minimize the chances of introducing errors, making the system easier to maintain.
- **Enhanced Flexibility:** New features can be added quickly without disrupting the existing functionality, allowing the software to adapt to new requirements.
- **Better Code Reusability:** Encourages the use of polymorphism and interfaces, promoting reusable components.

Here's a small example demonstrating the **Open/Closed Principle (OCP)** in Java. In this example, we'll start with code that violates OCP and then refactor it to follow the principle.

Scenario: Area Calculation for Shapes

Suppose you have a program that calculates the area of different shapes. Initially, this code does not follow OCP because every time we add a new shape, we must modify the `AreaCalculator` class.

Code that Violates OCP

```
class Rectangle {
    double width;
    double height;

    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
}

class Circle {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }
}

class AreaCalculator {
    public double calculate(Object shape) {
        if (shape instanceof Rectangle) {
            Rectangle rectangle = (Rectangle) shape;
            return rectangle.width * rectangle.height;
        } else if (shape instanceof Circle) {
            Circle circle = (Circle) shape;
            return Math.PI * circle.radius * circle.radius;
        }
        return 0;
    }
}
```

Problem: Each time a new shape is added (e.g., `Triangle`), we have to modify `AreaCalculator` to handle the new shape. This violates the Open/Closed Principle.

Refactored Code that Follows OCP

Now, let's refactor the code so that `AreaCalculator` doesn't need to change when a new shape is added. We can achieve this by using polymorphism.

```
// Step 1: Create a Shape interface
interface Shape {
    double calculateArea();
}

// Step 2: Implement each shape separately
class Rectangle implements Shape {
    double width;
    double height;

    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double calculateArea() {
        return width * height;
    }
}

class Circle implements Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}
```

```
// Step 3: Update AreaCalculator to use the Shape interface
class AreaCalculator {
    public double calculate(Shape shape) {
shapes    return shape.calculateArea(); // No need to modify this method for new
    }
}
}
```

Explanation

1. **Open for Extension:** We can add new shapes, like `Triangle`, by simply implementing the `Shape` interface and defining the `calculateArea` method. This doesn't require any changes to the `AreaCalculator` class.
2. **Closed for Modification:** `AreaCalculator` no longer needs to know the details of each shape. It simply relies on the `calculateArea` method, making the code more maintainable and extensible.

Usage Example

```
public class Main {
    public static void main(String[] args) {
        Shape rectangle = new Rectangle(5, 10);
        Shape circle = new Circle(7);

        AreaCalculator calculator = new AreaCalculator();

        System.out.println("Rectangle Area: " + calculator.calculate(rectangle));
        System.out.println("Circle Area: " + calculator.calculate(circle));
    }
}
```

Now, if you want to add a `Triangle` shape, you only need to create a new `Triangle` class that implements the `Shape` interface, without touching `AreaCalculator`. This approach adheres to the Open/Closed Principle.

Composition Over Inheritance

Composition over Inheritance is a fundamental design principle in object-oriented programming. It advocates building software by composing behavior from independent, reusable components rather than relying on class hierarchies through inheritance. This approach promotes greater flexibility, modularity, and maintainability in software systems.

Key Concepts of Composition Over Inheritance:

Composition:

Composition involves assembling objects with distinct responsibilities to build complex functionality. Instead of extending a base class, a class can **contain** instances of other classes and delegate work to them.

Inheritance:

Inheritance enables a class to inherit behavior from a parent class. While useful in some cases, overusing inheritance can lead to rigid and tightly coupled systems, especially when dealing with complex hierarchies.

Benefits of Composition Over Inheritance

Greater Flexibility:

Behavior can be easily swapped or modified at runtime without altering class hierarchies. This enables developers to build systems that are adaptable to change.

Improved Reusability:

Components can be reused across multiple contexts without being bound to a specific inheritance tree. This leads to cleaner, more modular designs.

Better Encapsulation:

Composition encourages the separation of concerns. Each component handles a specific responsibility, reducing the chance of side effects when making changes.

Avoidance of Fragile Base Class Problem:

Inheritance can create hidden dependencies on base class behavior. Composition avoids this issue by minimizing shared state and behavior.

When to Use Composition

- When different classes share behavior but do not form a strict "is-a" relationship.
- When you anticipate frequent changes to behavior or business rules.
- When aiming for modular, plug-and-play components.

By favoring composition over inheritance, developers can build more robust and maintainable software systems that are easier to extend, test, and refactor over time.

Readability

Some Concepts of Readable Code

- **Meaningful Names:**
 - Use descriptive and meaningful names for variables, functions, and classes.
 - Avoid using one-letter variables or non-descriptive names.
 - Function and class names should clearly convey their purpose.
- **Functions Should Be Small:**
 - Keep functions short and focused on doing one thing.
 - A function should only do one thing and do it well.
 - Avoid long parameter lists — if you have too many, consider refactoring.
- **One Level of Abstraction per Function:**
 - within a single function or method, all statements should operate at roughly the same level of abstraction
 - Mixing high-level concepts with low-level implementation details inside the same method makes the code harder to read and understand.

Test-Driven Development (TDD)

Test-Driven Development (TDD) is a software development methodology that emphasizes writing tests before writing the actual code that needs to be tested. It helps ensure that your code is reliable and meets the requirements from the very beginning. Here's a simple explanation suitable for junior developers:

Key Concepts of TDD

1. **Red-Green-Refactor Cycle:**
 - **Red:** Write a test for a new feature or functionality. This test should initially fail because the feature isn't implemented yet. This failure is a signal that you need to write the code to make the test pass.
 - **Green:** Write the minimum amount of code necessary to make the test pass. Focus on getting the test to succeed without worrying about optimization or code quality at this stage.
 - **Refactor:** Once the test is passing, improve the code structure and readability without changing its behavior. Ensure that all tests still pass after refactoring.
2. **Write Tests First:** Before writing the code for a new feature, you first write a test that describes how that feature should behave. This helps clarify the requirements and expected behavior before you start coding.
3. **Keep Tests Small and Focused:** Each test should check a specific piece of functionality. This makes it easier to identify which part of the code needs to be fixed if a test fails.

Benefits of TDD

- **Improved Code Quality:** Since tests are written before the code, developers are encouraged to write cleaner, more modular, and easier-to-understand code.
- **Fewer Bugs:** Writing tests first helps catch errors early in the development process, reducing the likelihood of bugs in production.
- **Clear Requirements:** TDD forces developers to think through requirements and specifications before coding, leading to a better understanding of what needs to be built.
- **Confidence to Refactor:** With a comprehensive suite of tests, developers can refactor code with confidence, knowing that existing functionality is covered by tests.

Coding Kata: Schweizer Krankenversicherungs-Prämienrechner

Github repository with code starting point and kata readme.md
https://github.com/catarata02/css_kata_birthday_greetings

Ausgangssituation

Du arbeitest für ein Schweizer Krankenversicherungsunternehmen. Deine Aufgabe ist es, ein System zu entwickeln, das die **monatliche Versicherungsprämie** für eine versicherte Person berechnet. Die Berechnung beginnt mit einer einfachen Regel basierend auf dem Alter.

Ausgangspunkt

Gegeben sind die folgenden Ausgangsklassen:

```
// Klasse Person mit einem Konstruktor und einem Feld "age"
public class Person {
    private int age;

    public Person(int age) {
        this.age = age;
    }

    public int getAge() {
        return age;
    }
}

public abstract class PremiumCalculator {
    public abstract double calculate(Person person);
}
```

Implementiere die Prämienberechnung basierend auf dem Alter:

- Unter 18: CHF 120
- 18–25: CHF 150
- 26–45: CHF 200
- 46–65: CHF 250
- Über 65: CHF 300

Anforderung: Die Struktur soll Erweiterungen ermöglichen, ohne dass bestehende Klassen geändert werden müssen. Beachte dabei das **Open/Closed Principle**.



Features – Schrittweise Erweiterung

Jedes Feature soll **ohne Modifikation der bestehenden Klassen** hinzugefügt werden. Ziel ist es, das System **durch Erweiterung statt Änderung** weiterzuentwickeln.



Feature 1: Geschlechtsanpassung

- Frauen erhalten **5 % Rabatt**.
- Männer zahlen den **Standardpreis**.

● Feature 2: Kantonaler Zuschlag

Schweizer Kantone haben unterschiedliche Gesundheitskosten:

- **Zürich (ZH):** +CHF 20
- **Genf (GE):** +CHF 25
- **Appenzell (AI):** -CHF 10
- **Andere Kantone:** +CHF 5

● Feature 3: Unfallversicherung

- **Standard:** Unfallversicherung eingeschlossen
- **Optional:** Kunde kann Unfall ausschliessen (**-10 % Rabatt**)

Feature 4: Zusatzversicherungen

Ein Kunde kann folgende Zusatzdeckungen wählen:

- Zahnversicherung: +CHF 30
- Sehhilfe: +CHF 15
- Komplementärmedizin: +CHF 40

Mehrere Optionen können kombiniert werden.

● Feature 5: Einkommen & Prämienverbilligung (KVG)

- Einkommen < **CHF 30'000**: -15 % Rabatt
- Einkommen **CHF 30'000–50'000**: -5 % Rabatt
- Einkommen > **CHF 50'000**: kein Rabatt

● **Feature 6: Familienrabatt**

- Wenn **zwei oder mehr Personen** derselben Familie versichert sind, erhalten alle **CHF 20 Rabatt**.

● Feature 7: Zahlungs-Treuebonus

- Kunden, die in den letzten **12 Monaten pünktlich gezahlt haben**, erhalten **CHF 10 Rabatt**.

● Feature 8: Risikokategorie

Je nach Gesundheitszustand oder Beruf wird eine Risikostufe zugewiesen:

- Geringes Risiko: **×1.0**
- Mittleres Risiko: **×1.1**
- Hohes Risiko: **×1.3**

Neue Risikostufen sollen einfach hinzufügbare sein.